DÉMO 002-B · CODE COMPLET

Babel, du JSX au JavaScript

Tous les fichiers utilisés, en entier, dans l'ordre de lecture.

Deux façons d'écrire le même composant : en JSX (converti par Babel) et à la main avec React.createElement. script.html : la page vide pour coder en live.

FICHIER **index.html** 52 lignes

```
<!--
  Démo 002-B · Babel : du JSX au JavaScript
  But : montrer que le JSX n'est qu'une écriture plus simple de React.createElement.
  Le navigateur ne comprend pas le JSX : Babel le traduit en JavaScript avant de l'exécuter.
  Ouvrir : double-clic sur ce fichier (pas besoin de npm ni de Vite). Il faut du réseau (scripts chargés depuis unpkg.com).
-->

<!-- La boîte vide où React va afficher le composant (comme la div "root" d'un projet Vite) -->
<div id="app"></div>

<!-- React : la bibliothèque -->
<script src="https://unpkg.com/react@18/umd/react.development.js" crossorigin></script>
<!-- ReactDOM : la partie qui affiche React dans la page (createRoot, render) -->
<script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js" crossorigin></script>
<!-- Babel directement dans le navigateur : il traduit le JSX en JavaScript à l'ouverture de la page -->
<script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>

<!-- type="text/babel" : « navigateur, ne lis pas ce code toi-même, laisse Babel le traduire d'abord » -->
<script type="text/babel">
  // 1) Version JSX (sera convertie par Babel)
  // C'est comme ça qu'on écrit tous les jours en React.
  function ComponentA() {
    return (
      <div>
        <h2 className="title">Hello World</h2>
      </div>
    );
  }


  // 2) Version sans JSX (on écrit React.createElement à la main)
  // C'est exactement ce que Babel produit à partir de ComponentA.
  // React.createElement(type, props, enfants) :
  //   - "div"  : la balise
  //   - null   : pas de props
  //   - 3e argument : l'enfant (ici un h2 avec la classe "title" et le texte "Hello World")
  function ComponentB() {
    return React.createElement(
      "div",
      null,
      React.createElement("h2", { className: "title" }, "Hello World")
    );
  }

  // On récupère la div "app" et on crée la racine React dedans
  const root = ReactDOM.createRoot(document.getElementById("app"));
  // Choisis UNE des deux lignes pour montrer la différence :
```

```
  // root.render(<ComponentA />);   // JSX → Babel → createElement (en coulisse)
  root.render(<ComponentB />);
  // Les deux affichent exactement la même chose à l'écran.

</script>
```

FICHIER **script.html** 19 lignes

```
<!--
  Démo 002-B · Babel · page de départ vide, pour coder en direct.
  React, ReactDOM et Babel sont déjà chargés : il reste à écrire le composant
  dans le <script type="text/babel"> ci-dessous, puis à l'afficher dans la div "app".
  La version complète, commentée, est dans index.html.
-->

<!-- La boîte vide où React va afficher le composant -->
<div id="app"></div>

<!-- React, ReactDOM (affichage dans la page) et Babel (traduction du JSX) -->
<script src="https://unpkg.com/react@18/umd/react.development.js" crossorigin></script>
<script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js" crossorigin></script>
<script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>

<!-- Babel traduit le code de ce script avant que le navigateur l'exécute -->
<script type="text/babel">

</script>
```